



Week 02 — Probability Distributions & the Central Limit Theorem

Bernoulli and Binomial trials, the Normal distribution, standardization, and why sample means behave predictably

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-09-17

Topics:

Package management	2
Where we left off	2
Reproducibility: <code>set.seed()</code>	2
Discrete distributions: the Bernoulli	3
The Binomial distribution	4
Example: will the summit venue overflow?	8
Continuous distributions: the Normal	9
Central Limit Theorem	13
Normal approximation of the Binomial	25
Variable standardization	28
Try it: a tighter invitation cap	31
Try it: standardizing a small conferences tibble	32
Try it: CLT at a larger sample size	32

Package management

Same as every session: bootstrap pacman, then load the tidyverse.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse)
```

Where we left off

Week 1 covered descriptive statistics (mean, median, variance, standard deviation, IQR) and a first pass at `ggplot2` on the Bundestag polling data. The lecture has since introduced probability distributions formally — we pick up straight from there and build those distributions in R, sticking to tidyverse throughout.

Reproducibility: `set.seed()`

Every `r`-prefixed distribution function (`rnorm()`, `rbinom()`, `rgamma()`, `sample()`, ...) draws an actual random sample, so it returns a different result on every call. Their `d/p/q`-prefixed counterparts (`dnorm()`, `pbinom()`, `qgamma()`, ...) instead compute a value straight from the distribution's formula — deterministic, with no variance in the output no matter how many times they're called. Random draws differ every run...

```
a <- rnorm(5)
b <- rnorm(5)
identical(a, b)
```

> OUTPUT

[1] FALSE

...unless we fix the seed right before each draw. This matters for homework and replicability in general — worth knowing before the many random draws coming up in this session.

```
set.seed(1234)
a <- rnorm(5)
set.seed(1234)
b <- rnorm(5)
identical(a, b)
```

> OUTPUT

[1] TRUE

In practice, `set.seed()` is usually called once, at the very top of a script, rather than before every draw — that alone makes the whole script’s random output reproducible top to bottom. Reseeding before each individual step, like above, mainly matters when coding interactively and re-running chunks out of order.

Discrete distributions: the Bernoulli

A Bernoulli trial is a single yes/no event (playground: probabilityplayground.com/bernoulli). Imagine one legislator voting on a bill, with a pre-vote whip count putting the probability of a “yes” vote at 0.8:

$$P(K = 1) = 0.8$$

```
dbinom(x = 1, size = 1, prob = 0.8) # yes vote
```

> OUTPUT

```
[1] 0.8
```

And therefore a “no” vote at:

$$P(K = 0) = 1 - 0.8 = 0.2$$

```
dbinom(x = 0, size = 1, prob = 0.8) # no vote
```

> OUTPUT

```
[1] 0.2
```

- `x` = number of successes (here: a “yes” vote)
- `size` = number of trials (1 legislator voting once)
- `prob` = probability of success

`size = 1` is exactly a Bernoulli trial, so it’s convenient to wrap `dbinom()` in a dedicated helper — same computation, but the function name now matches the distribution we mean. From here on, every Bernoulli probability uses `dbern()` instead of spelling out `dbinom(size = 1, ...)`. The math doesn’t change, only the function name — same two values, first the “yes” case:

$$P(K = 1) = 0.8$$

```
dbern <- function(x, prob) {  
  dbinom(x, size = 1, prob = prob)  
}  
  
dbern(x = 1, prob = 0.8) # same as dbinom(x = 1, size = 1, prob = 0.8)
```

> OUTPUT

```
[1] 0.8
```

Then the “no” case:

$$P(K = 0) = 0.2$$

```
dbern(x = 0, prob = 0.8) # same as dbinom(x = 0, size = 1, prob = 0.8)
```

> OUTPUT

```
[1] 0.2
```

Simulating one legislator's vote: the same shortcut applies to random draws — `rbern()` wraps `rbinom()` exactly the way `dbern()` wraps `dbinom()`. One simulated vote is itself a single random draw from that same distribution:

$$K \sim \text{Bernoulli}(p)$$

```
rbern <- function(n, prob) {
  rbino(n, size = 1, prob = prob)
}

rbern(n = 1, prob = 0.5) # same as rbino(n = 1, size = 1, prob = 0.5)
```

> OUTPUT

```
[1] 1
```

And 1000 independent single-legislator votes:

$$K_1, \dots, K_{1000} \stackrel{\text{iid}}{\sim} \text{Bernoulli}(p)$$

```
votes <- tibble(vote = rbern(n = 1000, prob = 0.5))
```

Tallied with `count()` instead of `table()` — same tibble-first approach as Week 1's mode section. This is the *empirical* count behind each outcome, the sample-based analogue of the PMF we computed by hand above:

```
votes |> count(vote)
```

> OUTPUT

```
# A tibble: 2 x 2
  vote     n
  <int> <int>
1     0  486
2     1  514
```

The Binomial distribution

Two legislators voting gives a Binomial with `size = 2` — the sample space is {yes-yes, yes-no, no-yes, no-no}. In general, for n trials at success probability p , the number of successes K has probability mass function

$$f^{\text{Bin}}(k; n, p) = P(K = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

— exactly what `dbinom()` computes. Every probability distribution in R follows this same **d/p/q/r** naming pattern — once you know what each prefix means for one distribution, you know it for all of them (playground: probabilityplayground.com/binomial):

Prefix	Concept	Formula	R $\leftrightarrow$ math parameters	What it gives you
d	Probability mass at one outcome (PMF)	$P(K = k; n, p) = \binom{n}{k} p^k (1-p)^{n-k}$	x = k , size = n , prob = p	The one point of the PMF at k
p	Cumulative probability up to an outcome (CDF)	$P(K \leq k; n, p) = \sum_{i=0}^k P(K = i; n, p)$	q = k , size = n , prob = p	Everything at or below k , summed
q	Inverse of p : outcome at a target cumulative probability	$q(\alpha; n, p) = \min\{k : P(K \leq k; n, p) \geq \alpha\}$	p = α , size = n , prob = p	Which k first reaches α
r	Random draws from the distribution	$K_1, \dots, K_m \stackrel{\text{iid}}{\sim} \text{Binomial}(n, p)$	n = m , size = n , prob = p	Actual simulated outcomes, not probabilities

Two argument names are worth double-checking every time: `qbinom()`’s first argument is **p** — the *target cumulative probability*, not the distribution’s own success probability (still supplied via **prob**) — and `rbinom()`’s first argument is **n** — *how many draws to generate*, not the distribution’s number of trials (still **size**).

Scaling up to a 100-member chamber at **p = 0.5** (a toss-up vote), the PMF and CDF over every possible yes-vote count:

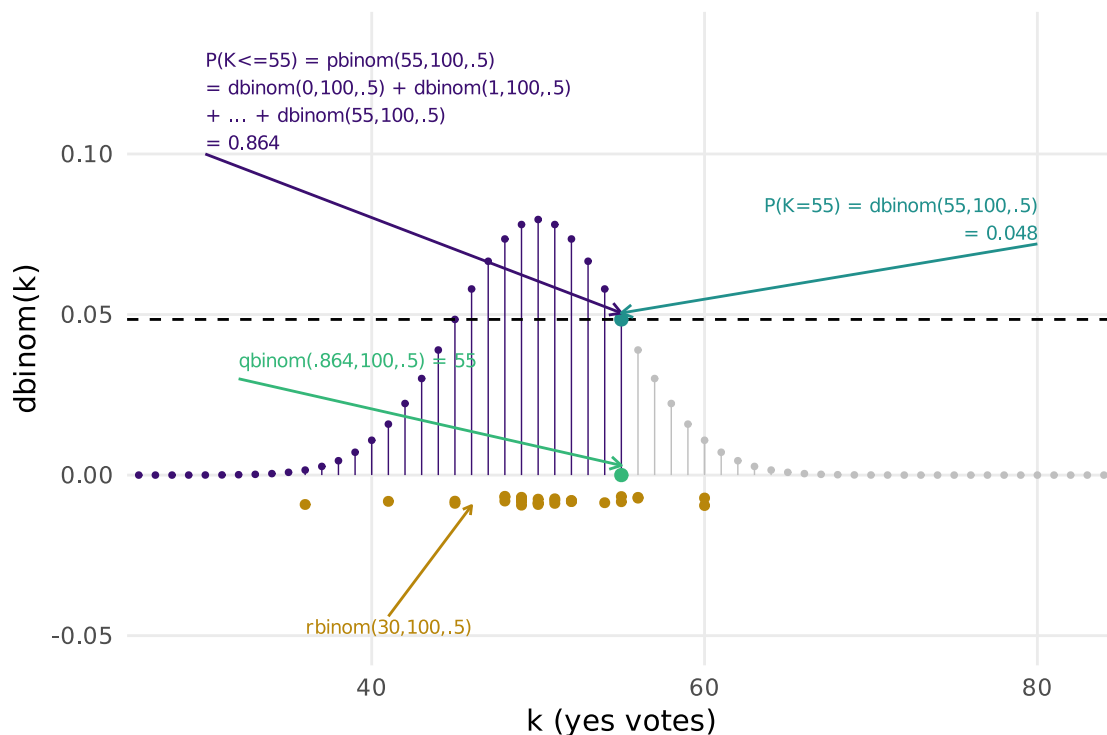
```
chamber_votes <- tibble(
  yes_votes = 0:100,
  pmf       = dbinom(yes_votes, size = 100, prob = 0.5),
  cdf       = pbinom(yes_votes, size = 100, prob = 0.5)
)
chamber_votes
```

```
> OUTPUT
```

```
# A tibble: 101 x 3  
  yes_votes      pmf      cdf  
    <int>     <dbl>   <dbl>  
1       0 0.000000000000000000000000000789 0.000000000000000000000000000789  
2       1 0.000000000000000000000000000789 0.000000000000000000000000000797  
3       2 0.000000000000000000000000000390 0.000000000000000000000000000398  
4       3 0.000000000000000000000000000128 0.000000000000000000000000000132  
5       4 0.000000000000000000000000000309 0.000000000000000000000000000322  
6       5 0.000000000000000000000000000594 0.000000000000000000000000000626  
7       6 0.000000000000000000000000000940 0.00000000000000000000000000100  
8       7 0.00000000000000000000000000126 0.00000000000000000000000000136  
9       8 0.00000000000000000000000000147 0.00000000000000000000000000160  
10      9 0.00000000000000000000000000150 0.00000000000000000000000000166  
# i 91 more rows
```

`dbinom()`, `pbinom()`, `qbinom()` and `rbinom()` all at once, visually: `dbinom()` traces the bars (we never connect them with a line — the outcome space is discrete, there is no such thing as 2.5 yes votes), `pbinom()` is the highlighted area's running total, `qbinom()` finds the boundary from a cumulative probability, and `rbinom()` draws actual random outcomes. `k_ref = 55` here is just an arbitrary reference point to show all four at once.

K ~ Binomial(100, .5): d/p/q/r at a glance



Zooming into a small, hand-checkable case — two legislators, `size = 2`. Each possible vote count has its own PMF value:

$$P(K = k) = \binom{2}{k} 0.5^k (1 - 0.5)^{2-k}$$

```
dbinom(x = c(0, 1, 2), size = 2, prob = 0.5) # exactly 0, 1, 2 yes votes
```

> OUTPUT

```
[1] 0.25 0.50 0.25
```

How likely is at most one yes vote?

$$P(K \leq 1) = \sum_{i=0}^1 P(K = i)$$

```
pbinom(q = 1, size = 2, prob = 0.5) # 0 or 1 yes votes
```

> OUTPUT

```
[1] 0.75
```

`pbinom()` is the CDF, so the probability of *more than* 1 yes vote is either `1 - that`, or `lower.tail = FALSE`:

$$P(K > 1) = 1 - P(K \leq 1)$$

```
1 - pbinom(q = 1, size = 2, prob = 0.5)
```

> OUTPUT

```
[1] 0.25
```

```
pbinom(q = 1, size = 2, prob = 0.5, lower.tail = FALSE)
```

> OUTPUT

```
[1] 0.25
```

Which vote count corresponds to the 75th percentile of outcomes?

$$q(\alpha) = \min\{k : P(K \leq k) \geq \alpha\}, \quad \alpha = 0.75$$

```
qbinom(p = 0.75, size = 2, prob = 0.5) # smallest k with P(K<=k) >= .75
```

> OUTPUT

```
[1] 1
```

Five simulated two-legislator outcomes:

$$K_1, \dots, K_5 \stackrel{\text{iid}}{\sim} \text{Binomial}(2, 0.5)$$

```
rbinom(n = 5, size = 2, prob = 0.5) # 5 simulated two-legislator outcomes
```

>_ OUTPUT

```
[1] 1 0 0 0 1
```

Example: will the summit venue overflow?

A summit chair invited 120 delegates, each independently having an 85% chance of actually showing up. The venue comfortably seats 100. What's the probability that *more than* 100 delegates actually show up?

$$P(K > 100) = 1 - P(K \leq 100), \quad K \sim \text{Binomial}(120, 0.85)$$

```
1 - pbinom(q = 100, size = 120, prob = 0.85)
```

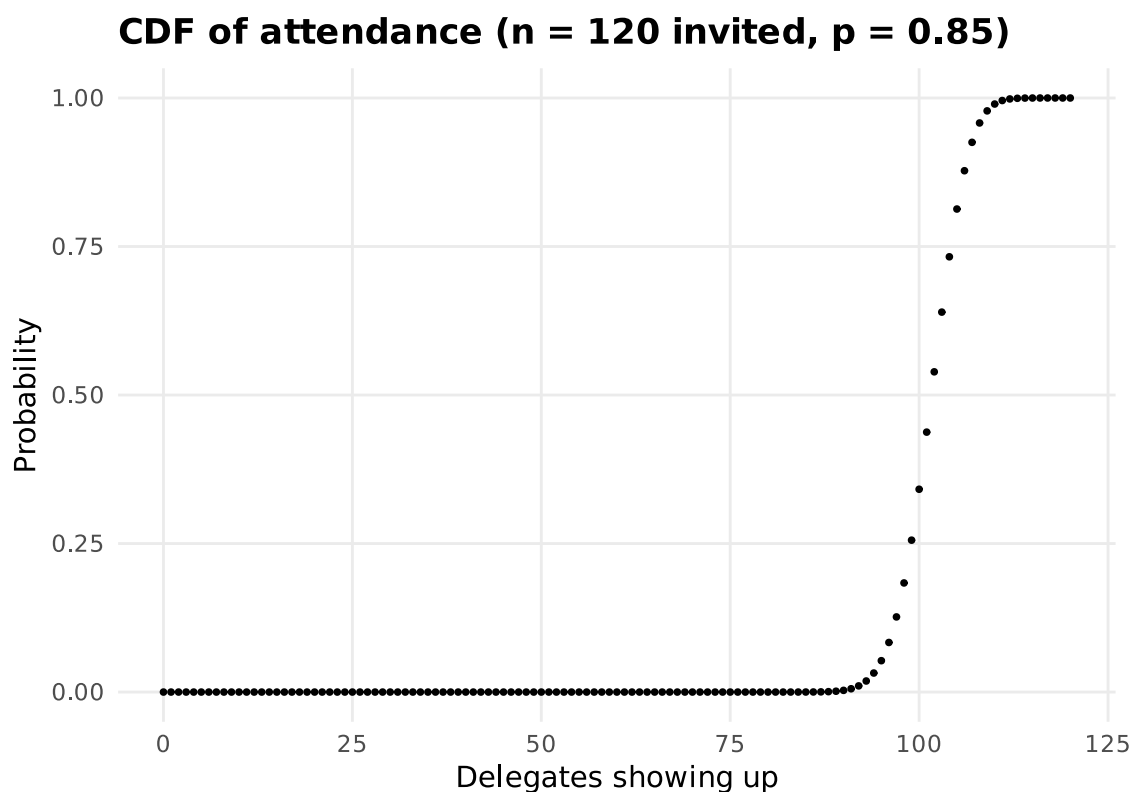
>_ OUTPUT

```
[1] 0.6586167
```

This is the same “more than 100 people show up” question as a binomial process — 120 independent trials, each showing up with probability 0.85. Plotting the CDF across every possible attendance count:


```
summit_cdf <- tibble(
  attendees = 0:120,
  cdf       = pbinom(attendees, size = 120, prob = 0.85)
)

summit_cdf |>
  ggplot(aes(x = attendees, y = cdf)) +
  geom_point(size = 0.8) +
  labs(
    title = "CDF of attendance (n = 120 invited, p = 0.85)",
    x     = "Delegates showing up",
    y     = "Probability"
  ) +
  theme_intror()
```



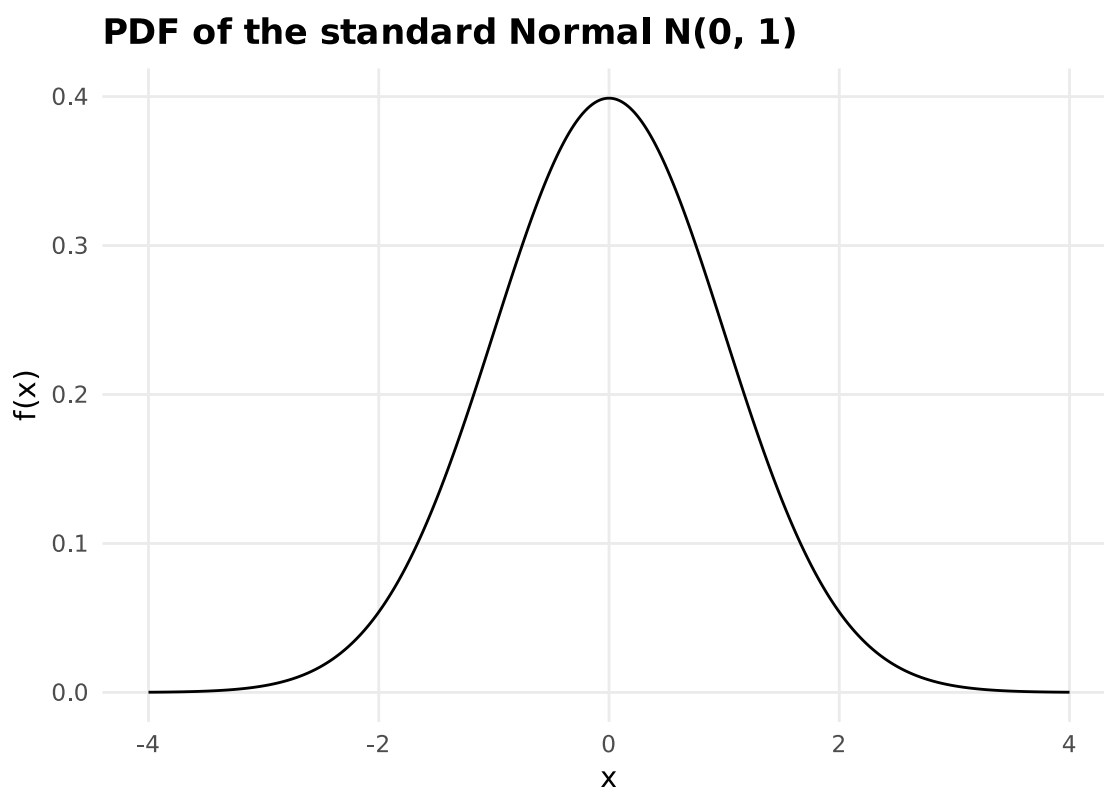
Continuous distributions: the Normal

The Normal's four prefixes — **d**, **p**, **q**, **r** — work exactly like Binomial's, just with a continuous x in place of a discrete k (playground: probabilityplayground.com/normal). Its PDF and CDF:

$$f^{\text{Norm}}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

$$F^{\text{Norm}}(x; \mu, \sigma^2) = \int_{-\infty}^x f^{\text{Norm}}(t; \mu, \sigma^2) dt = \Phi\left(\frac{x - \mu}{\sigma}\right)$$

```
tibble(
  x      = seq(-4, 4, by = 0.01),
  density = dnorm(x)
) |>
ggplot(aes(x = x, y = density)) +
  geom_line() +
  labs(
    title = "PDF of the standard Normal N(0, 1)",
    x      = "x",
    y      = "f(x)"
  ) +
  theme_intror()
```



Inverting the CDF returns an x -value instead of a probability. Worth anchoring the walkthrough on a value you already know from countless 95%-CI calculations — “1.96” is nothing more than `qnorm()`’s answer to “which x has 97.5% of the mass below it?”, worth deriving and storing instead of memorizing:

$$q(0.975) = \Phi^{-1}(0.975)$$

```
crit_975 <- qnorm(p = 0.975, mean = 0, sd = 1)
crit_975
```

> OUTPUT

```
[1] 1.959964
```

Reading probabilities off that curve — the chance of landing at or below that same value:

$$P(X \leq 1.96)$$

```
pnorm(q = crit_975, mean = 0, sd = 1)
```

> OUTPUT

```
[1] 0.975
```

And its complement, landing above it:

$$P(X > 1.96) = 1 - P(X \leq 1.96)$$

```
1 - pnorm(q = crit_975, mean = 0, sd = 1)
```

> OUTPUT

```
[1] 0.025
```

`q()` isn't limited to 97.5% — any target probability works, e.g. which value has 30% of the mass below it?

$$q(0.3) = \Phi^{-1}(0.3)$$

```
qnorm(p = 0.3, mean = 0, sd = 1) # value below which 30% of the mass lies
```

> OUTPUT

```
[1] -0.5244005
```

And drawing 20 actual random values from the distribution:

$$X_1, \dots, X_{20} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1)$$

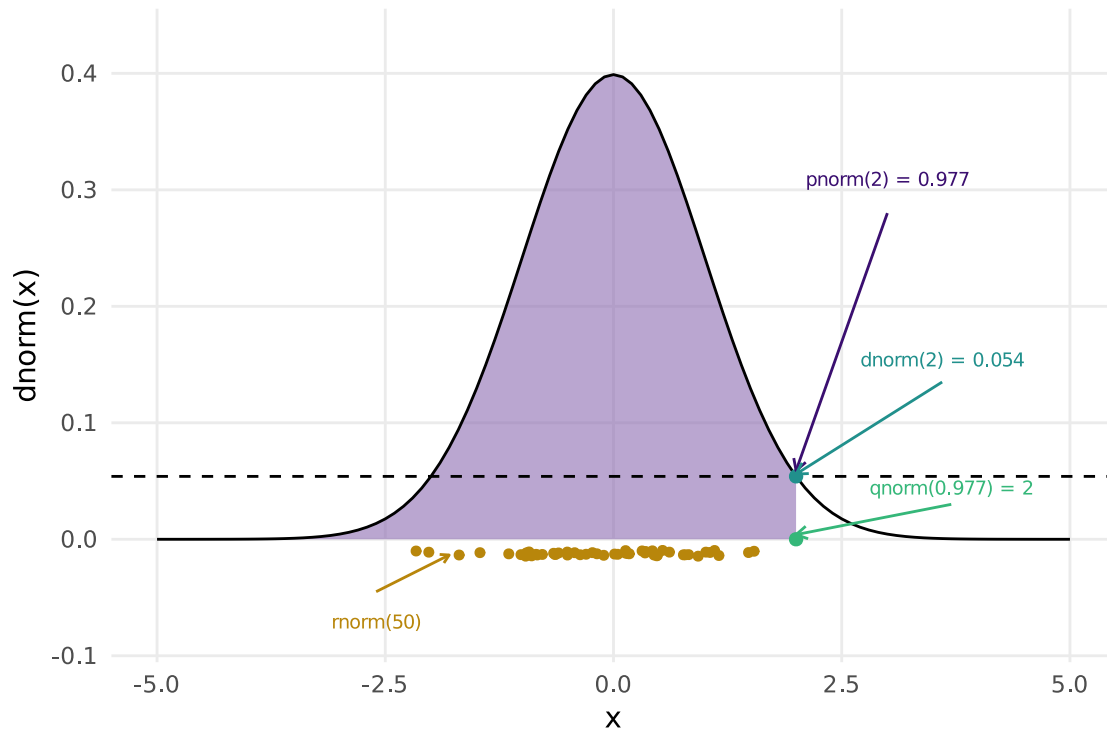
```
rnorm(n = 20, mean = 0, sd = 1)
```

> OUTPUT

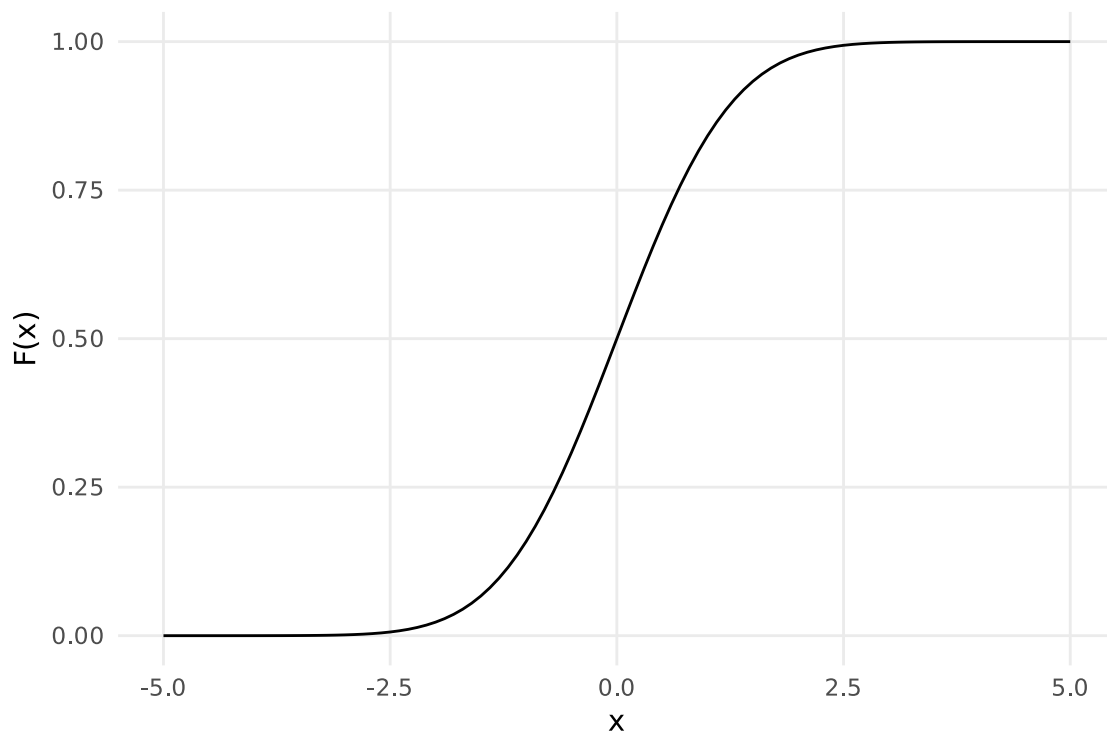
```
[1] -0.6886715  0.1701978  0.5944834 -0.8150327 -2.8643468  0.9538235  0.6152848
[2] -0.7008707 -1.0736613 -0.2999912
[11]  0.1081066  0.7232008  1.3900951 -0.8488834  0.2907228  1.3341760 -0.9013422
[20] -0.2227634 -1.7714680  0.2874223
```

Plotting the standard Normal's PDF and CDF — and `dbinom()/pbinom()/qbinom()/rbinom()`'s continuous counterparts, all at once: the shaded area is `pnorm()`, its boundary height is `dnorm()`, the boundary's x-position is `qnorm()`, and the orange dots below are actual `rnorm()` draws.

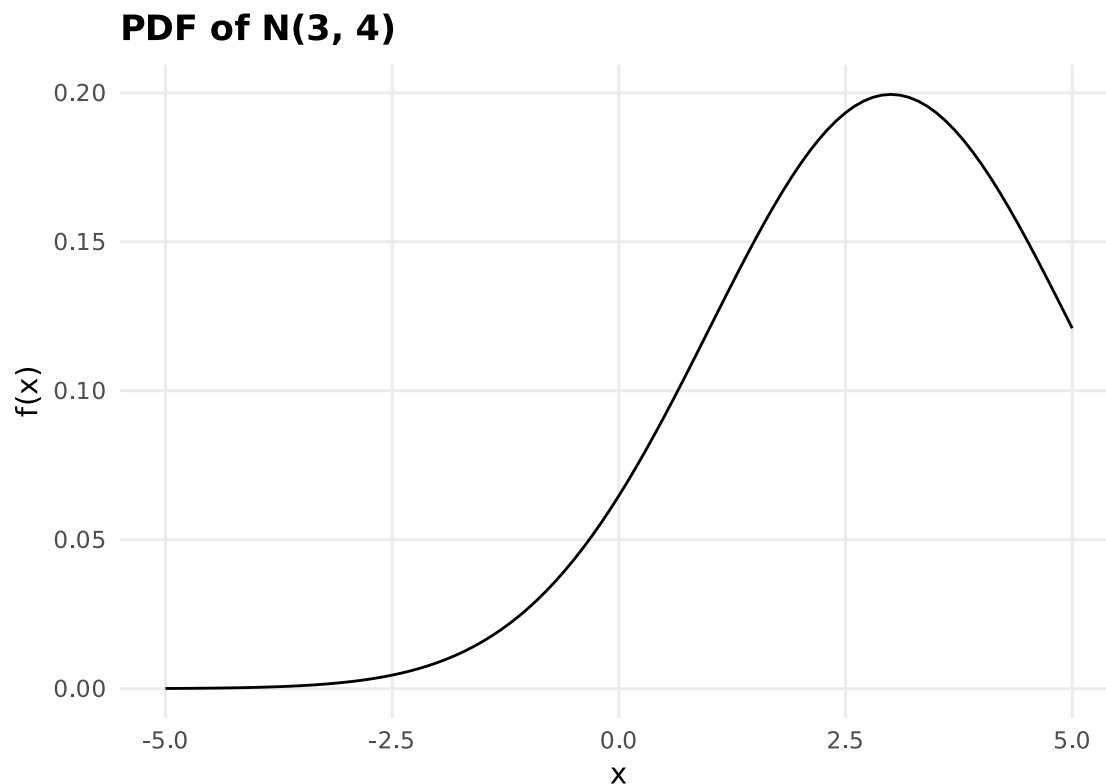
PDF for $X \sim N(0, 1)$: d/p/q/r at a glance



CDF of $N(0, 1)$



A different mean and sd shifts and stretches the same curve.



Central Limit Theorem

For a population with mean μ and variance σ^2 , the sampling distribution of the mean approaches $\mathcal{N}(\mu, \frac{\sigma^2}{n})$ as n grows — regardless of the population's own shape. Zooming out from any one summit: imagine every delegate who's ever attended one of these annual conferences, with a true average age of 32 (sd 5) — we never get this in practice, an organizer only ever surveys whoever's in the room at a given session. The trials below repeat that batch-survey process at different sample sizes and on two differently-shaped populations, to see how closely the simulated sampling distribution matches the CLT's prediction:

- **Trial 1** — delegate age (Normal population)
 - **1.a:** 100 surveys of 150 delegates each
 - **1.b:** 100 surveys of 400 delegates each
- **Trial 2** — travel distance to the conference (skewed population)
 - **2.a:** 100 surveys of 100 delegates each
 - **2.b:** 100 surveys of 400 delegates each

```
population_size <- 1000
n_batches <- 100
delegate_ages <- rnorm(population_size, mean = 32, sd = 5)

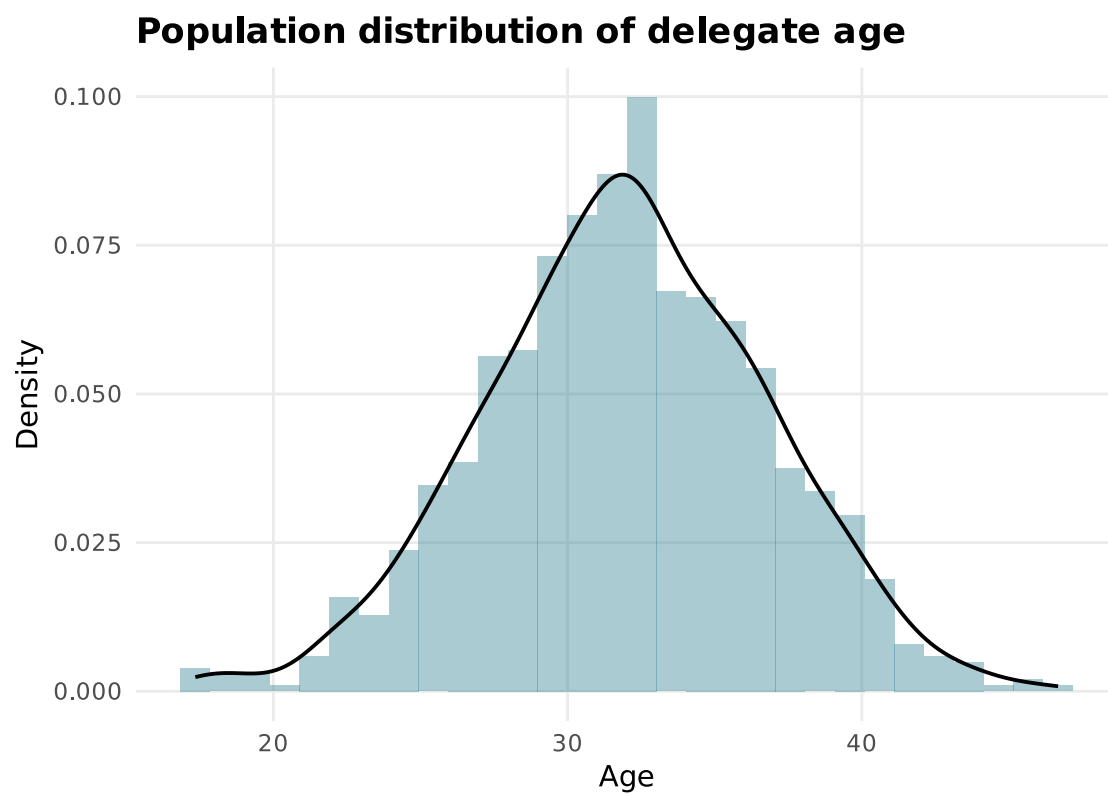
tibble(age = delegate_ages) |>
  summarize(
    mean_age = mean(age),
    var_age = var(age)
  )
```

> OUTPUT

```
# A tibble: 1 x 2
  mean_age var_age
  <dbl>    <dbl>
1    32.0    23.0
```

The population itself, histogram and density together:

```
tibble(age = delegate_ages) |>
  ggplot(aes(x = age)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_density(linewidth = 0.8) +
  labs(
    title = "Population distribution of delegate age",
    x      = "Age",
    y      = "Density"
  ) +
  theme_intror()
```



`slice_sample()` draws rows the tidyverse way — `n` = plus `replace` = mirrors `rbinom()`'s own arguments:

```
tibble(age = delegate_ages) |>
  slice_sample(n = 30, replace = TRUE)
```

> OUTPUT

```
# A tibble: 30 x 1
  age
<dbl>
1  37.1
2  38.9
3  23.4
4  26.8
5  33.0
6  32.9
7  35.9
8  40.6
9  36.7
10 27.4
# i 20 more rows
```

`delegate_ages` itself is kept as a plain vector rather than a tibble, though, so the trials below sample it with base R's `sample()` instead — `slice_sample()` draws data-frame *rows*, `sample()` draws *elements* of a vector.

An organizer can only ever survey a batch at a time, never the whole population. The next two trials repeat that batch-survey process `n_batches` times each, at two different survey sizes, to see how the *sampling distribution of the mean* behaves — **trial 1.a** surveys 150 delegates per batch, **trial 1.b** surveys 400. `expand_grid()` lays out every (batch, participant) combination up front, `group_by(batch)` then draws each batch's own resample within `mutate()` — no `for` loop needed, and each batch is printed in full first, before collapsing it down to sample means:


```

sample_size <- 150
trial_1a <- expand_grid(
  batch      = 1:n_batches,
  participant = 1:sample_size
) |>
  group_by(batch) |>
  mutate(age = sample(delegate_ages, size = sample_size, replace = TRUE)) |>
  ungroup() |>
  mutate(label = str_glue("1.a: n = {sample_size} per survey"))
trial_1a |> head(100)

```

> OUTPUT

```

# A tibble: 100 x 4
  batch participant    age label
  <int>      <int> <dbl> <glue>
1     1         1     20.3 1.a: n = 150 per survey
2     1         2     36.2 1.a: n = 150 per survey
3     1         3     31.7 1.a: n = 150 per survey
4     1         4     36.5 1.a: n = 150 per survey
5     1         5     31.8 1.a: n = 150 per survey
6     1         6     27.2 1.a: n = 150 per survey
7     1         7     32.3 1.a: n = 150 per survey
8     1         8     32.3 1.a: n = 150 per survey
9     1         9     45.5 1.a: n = 150 per survey
10    1        10     29.1 1.a: n = 150 per survey
# i 90 more rows

```

```

sample_size <- 400
trial_1b <- expand_grid(
  batch      = 1:n_batches,
  participant = 1:sample_size
) |>
  group_by(batch) |>
  mutate(age = sample(delegate_ages, size = sample_size, replace = TRUE)) |>
  ungroup() |>
  mutate(label = str_glue("1.b: n = {sample_size} per survey"))
trial_1b |> head(100)

```

> OUTPUT

```

# A tibble: 100 x 4
  batch participant   age label
  <int>      <int> <dbl> <glue>
1     1         1  34.0 1.b: n = 400 per survey
2     1         2  41.4 1.b: n = 400 per survey
3     1         3  35.1 1.b: n = 400 per survey
4     1         4  24.2 1.b: n = 400 per survey
5     1         5  31.7 1.b: n = 400 per survey
6     1         6  29.8 1.b: n = 400 per survey
7     1         7  28.2 1.b: n = 400 per survey
8     1         8  32.1 1.b: n = 400 per survey
9     1         9  33.1 1.b: n = 400 per survey
10    1        10  33.5 1.b: n = 400 per survey
# i 90 more rows

```

One sample mean per batch, for each survey size:

```

trials_1 <- bind_rows(trial_1a, trial_1b) |>
  group_by(batch, label) |>
  summarize(sample_mean = mean(age), .groups = "drop")

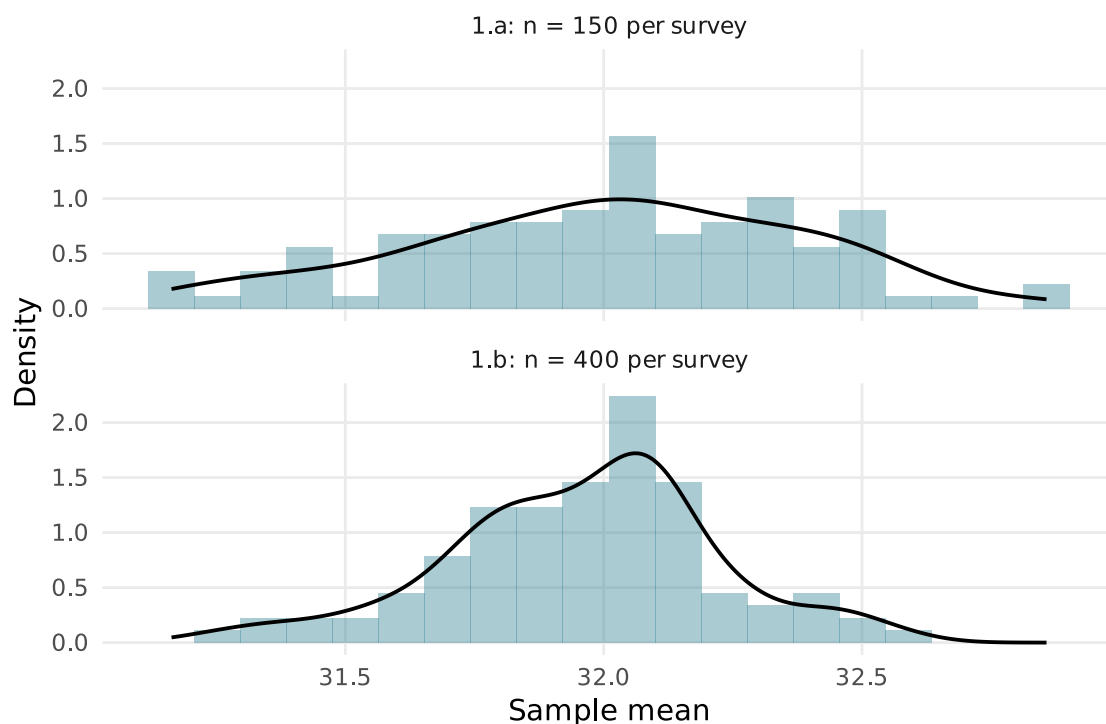
```

```

trials_1 |>
  ggplot(aes(x = sample_mean)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_density(linewidth = 0.8) +
  facet_wrap(~label, ncol = 1) +
  labs(
    title = "Sampling distribution of the mean, by survey size",
    x     = "Sample mean",
    y     = "Density"
  ) +
  theme_intror()

```

Sampling distribution of the mean, by survey size



Directly comparing both trials' actual mean/variance:

```

trials_1 |>
  group_by(label) |>
  summarize(
    mean_sm = mean(sample_mean),
    var_sm  = var(sample_mean)
  )

```

> OUTPUT

```

# A tibble: 2 x 3
  label                mean_sm var_sm
<glue>             <dbl>   <dbl>
1 1.a: n = 150 per survey 32.0 0.145
2 1.b: n = 400 per survey 32.0 0.0682

```

Against the CLT's own prediction, $\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$, for each batch size:

```
tibble(population_variance = var(delegate_ages)) |>
  mutate(
    theoretical_1a = population_variance / 150,
    theoretical_1b = population_variance / 400
  )
```

>_ OUTPUT

```
# A tibble: 1 x 3
  population_variance theoretical_1a theoretical_1b
      <dbl>          <dbl>          <dbl>
1      23.0          0.153          0.0574
```

Larger samples per survey (400 vs. 150) tighten the sampling distribution around the true population mean — exactly why pollsters with bigger samples give more precise estimates.

Does the CLT still hold for a population that isn't Normal at all? Delegates' travel distance to reach the conference is a good example: most attend from nearby, a few travel intercontinental — modeled with a Gamma distribution (playground: probabilityplayground.com/gamma), shape k controlling how peaked it is and scale θ stretching it out:

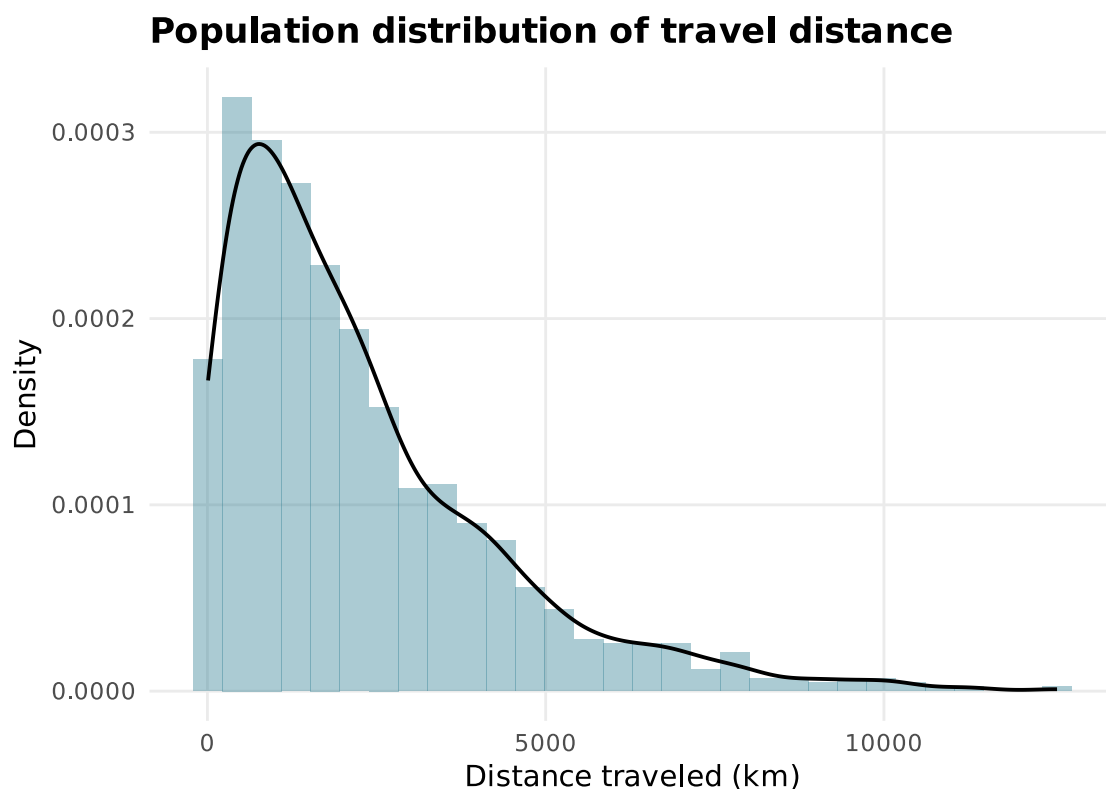
$$f^{\text{Gam}}(x; k, \theta) = \frac{1}{\Gamma(k) \theta^k} x^{k-1} e^{-\frac{x}{\theta}}, \quad k = 1.1, \theta = 2000$$

```

travel_distances <- rgamma(population_size, shape = 1.1, scale = 2000)

tibble(distance_km = travel_distances) |>
  ggplot(aes(x = distance_km)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_density(linewidth = 0.8) +
  labs(
    title = "Population distribution of travel distance",
    x     = "Distance traveled (km)",
    y     = "Density"
  ) +
  theme_intror()

```



Clearly right-skewed — nothing like a Normal. Same approach as before, just resampling `travel_distances` instead of `delegate_ages` — **trial 2.a** at 100 per survey, **trial 2.b** at 400:

```

sample_size <- 100
trial_2a <- expand_grid(
  batch      = 1:n_batches,
  participant = 1:sample_size
) |>
  group_by(batch) |>
  mutate(distance_km = sample(travel_distances, size = sample_size, replace = TRUE)) |>
  ungroup() |>
  mutate(label = str_glue("2.a: n = {sample_size} per survey"))
trial_2a |> head(100)

```

> OUTPUT

```

# A tibble: 100 x 4
  batch participant distance_km label
  <int>      <int>      <dbl> <glue>
1     1         1         307. 2.a: n = 100 per survey
2     1         2        1852. 2.a: n = 100 per survey
3     1         3        3518. 2.a: n = 100 per survey
4     1         4         269. 2.a: n = 100 per survey
5     1         5        2740. 2.a: n = 100 per survey
6     1         6        2049. 2.a: n = 100 per survey
7     1         7        5063. 2.a: n = 100 per survey
8     1         8        1160. 2.a: n = 100 per survey
9     1         9        2241. 2.a: n = 100 per survey
10    1        10        3378. 2.a: n = 100 per survey
# i 90 more rows

```

```

sample_size <- 400
trial_2b <- expand_grid(
  batch      = 1:n_batches,
  participant = 1:sample_size
) |>
  group_by(batch) |>
  mutate(distance_km = sample(travel_distances, size = sample_size, replace = TRUE)) |>
  ungroup() |>
  mutate(label = str_glue("2.b: n = {sample_size} per survey"))
trial_2b |> head(100)

```

> OUTPUT

```

# A tibble: 100 x 4
  batch participant distance_km label
  <int>      <int>      <dbl> <glue>
1     1         1      1428. 2.b: n = 400 per survey
2     1         2      1426. 2.b: n = 400 per survey
3     1         3      3113. 2.b: n = 400 per survey
4     1         4      2715. 2.b: n = 400 per survey
5     1         5      2376. 2.b: n = 400 per survey
6     1         6      1651. 2.b: n = 400 per survey
7     1         7       834. 2.b: n = 400 per survey
8     1         8      4934. 2.b: n = 400 per survey
9     1         9      1333. 2.b: n = 400 per survey
10    1        10      2623. 2.b: n = 400 per survey
# i 90 more rows

```

```

trials_2 <- bind_rows(trial_2a, trial_2b) |>
  group_by(batch, label) |>
  summarize(sample_mean = mean(distance_km), .groups = "drop")

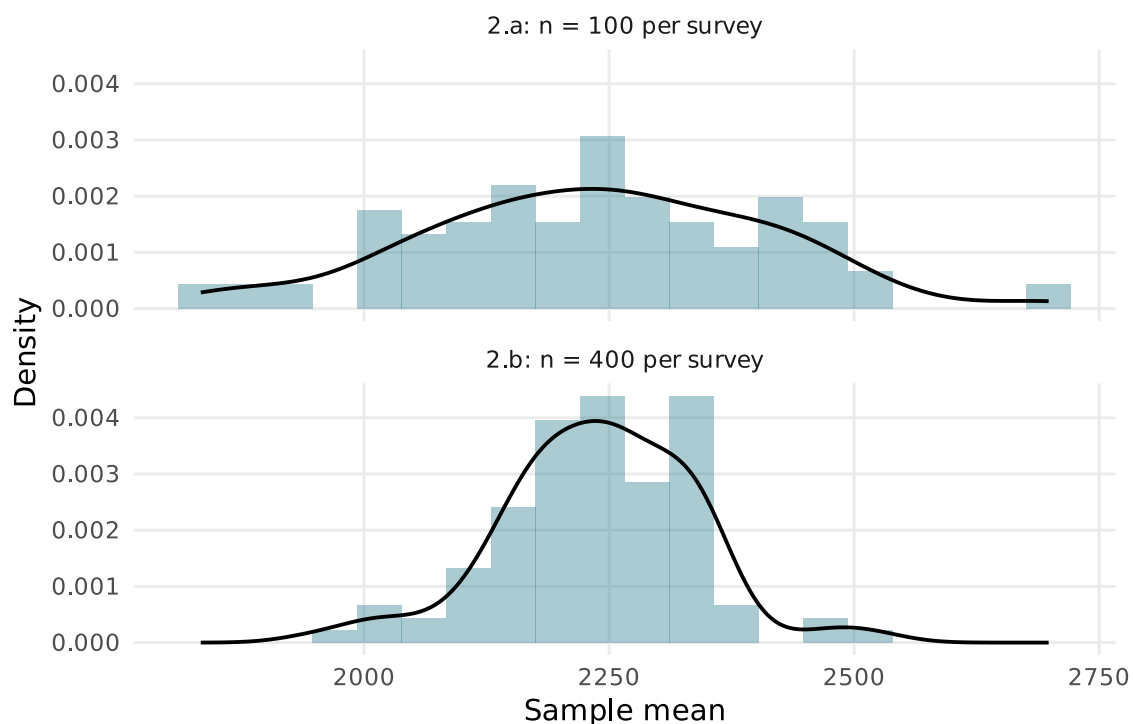
```

```

trials_2 |>
  ggplot(aes(x = sample_mean)) +
  geom_histogram(aes(y = after_stat(density)), bins = 20, fill = course_primary, alpha
= 0.4, color = NA) +
  geom_density(linewidth = 0.8) +
  facet_wrap(~label, ncol = 1) +
  labs(
    title = "Sampling distribution of the mean, skewed population",
    x     = "Sample mean",
    y     = "Density"
  ) +
  theme_intror()

```

Sampling distribution of the mean, skewed populat



Directly comparing both trials again:

```

trials_2 |>
  group_by(label) |>
  summarize(
    mean_sm = mean(sample_mean),
    var_sm  = var(sample_mean)
  )

```

> OUTPUT

```

# A tibble: 2 x 3
  label                mean_sm var_sm
<glue>              <dbl>   <dbl>
1 2.a: n = 100 per survey 2237. 30617.
2 2.b: n = 400 per survey 2239.  9757.

```


The sampling distribution of the mean approaches Normal regardless of the population's own shape — that's the whole point of the theorem.

Both trials above resampled a single variable at a time. Sampling several variables *together* — one delegate's age and travel distance drawn as a pair, not independently — preserves whatever relationship holds between them within a batch, the same way `slice_sample()` on a tibble resamples whole rows rather than each column separately:

```
delegates <- tibble(
  age      = rnorm(population_size, mean = 32, sd = 5),
  distance_km = rgamma(population_size, shape = 1.1, scale = 2000)
)
```

```
sample_size <- 100
trial_combined <- expand_grid(
  batch      = 1:n_batches,
  participant = 1:sample_size
) |>
  group_by(batch) |>
  mutate(sample = slice_sample(delegates, n = sample_size, replace = TRUE)) |>
  ungroup() |>
  unnest(sample) |>
  mutate(label = str_glue("Joint age + distance, n = {sample_size} per survey"))
trial_combined |> head(100)
```

> OUTPUT

```
# A tibble: 100 x 5
  batch participant   age distance_km label
  <int>      <int> <dbl>      <dbl> <glue>
1     1         1  31.4      1706. Joint age + distance, n = 100 per survey
2     1         2  34.8       773. Joint age + distance, n = 100 per survey
3     1         3  40.6      108. Joint age + distance, n = 100 per survey
4     1         4  38.3      561. Joint age + distance, n = 100 per survey
5     1         5  33.6      933. Joint age + distance, n = 100 per survey
6     1         6  33.7     1838. Joint age + distance, n = 100 per survey
7     1         7  39.6     2089. Joint age + distance, n = 100 per survey
8     1         8  30.2     1444. Joint age + distance, n = 100 per survey
9     1         9  37.5     1098. Joint age + distance, n = 100 per survey
10    1        10  37.4     2176. Joint age + distance, n = 100 per survey
# i 90 more rows
```

Normal approximation of the Binomial

A Binomial count is itself a sum of n independent Bernoulli trials — by the CLT we just saw, that sum's own distribution should look increasingly Normal as n grows, the same logic as a sampling distribution of a mean. That means a Binomial's probabilities can be approximated with a Normal distribution matched on mean and variance — useful whenever an exact `pbinom()` calculation would be impractical, and the reason this approximation is worth knowing rather than always just calling `pbinom()` directly. Back to the summit scenario:

$$E(K) = np, \quad \text{Var}(K) = np(1 - p)$$

```
n_invited <- 120
p_attend <- 0.85
mean_attend <- n_invited * p_attend
sd_attend <- sqrt(n_invited * p_attend * (1 - p_attend))
```

In general, the Normal approximation replaces an exact Binomial tail probability with the matching Normal's:

$$P(K > 100) \approx P(X > 100), \quad X \sim \mathcal{N}(np, np(1 - p))$$

Concretely, `pnorm()` computes that Normal probability directly off the Normal CDF itself — the same F^{Norm} from this session's Normal-CDF formula above:

$$P(X > 100) = 1 - F^{\text{Norm}}(100; np, np(1 - p))$$

```
1 - pnorm(q = 100, mean = mean_attend, sd = sd_attend)
```

> OUTPUT

```
[1] 0.695433
```

`pbinom()` computes that same right-tail probability exactly instead — straight off the Binomial CDF itself, no Normal approximation involved:

$$P(K > 100) = 1 - F^{\text{Bin}}(100; n, p)$$

```
1 - pbinom(q = 100, size = n_invited, prob = p_attend) # close to the normal approx above!
```

> OUTPUT

```
[1] 0.6586167
```

The two match closely — closely enough that overlaying the exact Binomial pmf (`dbinom()` at every integer k) directly on top of its fitted Normal curve — $\mathcal{N}(np, np(1 - p))$, evaluated at this scenario's own `mean_attend/sd_attend` — makes the approximation's quality visible at a glance: the same curve whose right tail beyond 100 `pnorm()` just integrated above.

```

binom_pmf <- tibble(
  attendees = 0:n_invited,
  pmf       = dbinom(attendees, n_invited, p_attend)
)

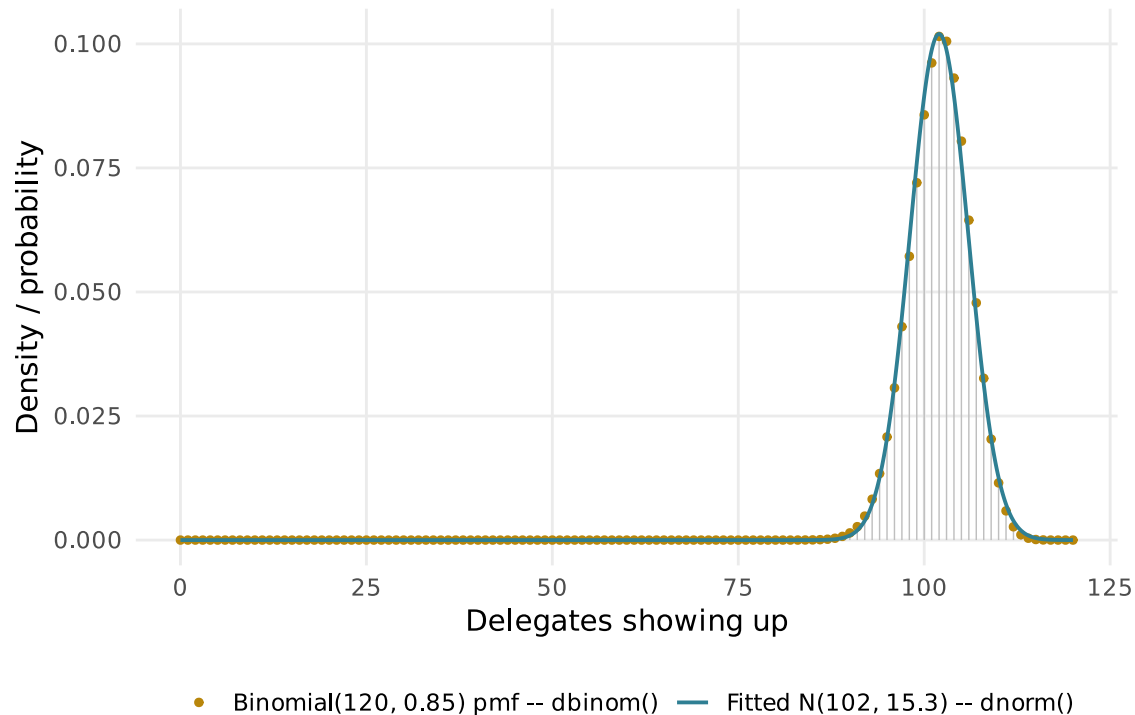
normal_pdf <- tibble(
  attendees = seq(0, n_invited, by = 0.01),
  density   = dnorm(attendees, mean_attend, sd_attend)
)

binom_label <- str_glue("Binomial({n_invited}, {p_attend}) pmf -- dbinom()")
normal_label <- str_glue("Fitted N({mean_attend}, {round(sd_attend^2, 1)}) -- dnorm()")

ggplot() +
  geom_segment(
    data = binom_pmf, aes(x = attendees, xend = attendees, y = 0, yend = pmf),
    color = "grey75", linewidth = 0.3
  ) +
  geom_point(data = binom_pmf, aes(x = attendees, y = pmf, color = binom_label), size
= 1.2) +
  geom_line(data = normal_pdf, aes(x = attendees, y = density, color = normal_label),
linewidth = 0.8) +
  scale_color_manual(name = NULL, values = set_names(c("#B8860B", course_primary),
c(binom_label, normal_label))) +
  labs(
    title = "Binomial pmf vs. its Normal approximation",
    x     = "Delegates showing up",
    y     = "Density / probability"
  ) +
  theme_intror()

```

Binomial pmf vs. its Normal approximation



Variable standardization

Comparing two conferences' attendance only makes sense relative to each conference series' own typical turnout — different conference series have wildly different baselines. The z-score formula standardizes any value against its own distribution's mean and sd:

$$z_i = \frac{x_i - \bar{x}}{\sigma}$$

Rather than typing each series' historical mean/sd in by hand, simulate a run of past years for each series and read `mean()/sd()` straight off the data — series A has historically run smaller (mean 60, sd 20), series B larger (mean 120, sd 40):

```
set.seed(1234)
series_a <- rnorm(1000, mean = 60, sd = 20)
series_b <- rnorm(1000, mean = 120, sd = 40)
```

Conference A drew 90 delegates this year — which of the two had the stronger year, relative to its own history? Standardized against series A's own `mean()/sd()`:

$$z_a = \frac{x_a - \bar{x}_a}{\sigma_a}$$

```

mean_a <- mean(series_a)
sd_a    <- sd(series_a)
conf_a_attendance <- 90

z_a <- (conf_a_attendance - mean_a) / sd_a
z_a

```

> OUTPUT

```
[1] 1.530672
```

Conference B drew 160 delegates this year. Standardized against series B's own `mean()/sd()`:

$$z_b = \frac{x_b - \bar{x}_b}{\sigma_b}$$

```

mean_b <- mean(series_b)
sd_b    <- sd(series_b)
conf_b_attendance <- 160

z_b <- (conf_b_attendance - mean_b) / sd_b
z_b

```

> OUTPUT

```
[1] 1.004381
```

Conference A comes out roughly 1.5 standard deviations above its own series' average, Conference B only about 1.0 — so A had the stronger year relative to its own history, despite drawing fewer delegates in absolute terms. (Only “roughly,” since `series_a/series_b` are themselves simulated samples — `mean()/sd()` only approximate the true 60/20 and 120/40 they were drawn from, same estimation idea as the CLT trials above.) Z-scores are directly interpretable as “how many standard deviations above/below the mean.” Visually, each conference sits within its *own* series' distribution — A's dashed line falls further into its own tail than B's, even on a smaller absolute scale. Each conference's own curve, built separately, then combined:

```

conf_a_curve <- tibble(
  x          = seq(mean_a - 3 * sd_a, mean_a + 3 * sd_a, length.out = 200),
  density    = dnorm(x, mean = mean_a, sd = sd_a),
  conference = "A",
  attendance = conf_a_attendance
)

conf_b_curve <- tibble(
  x          = seq(mean_b - 3 * sd_b, mean_b + 3 * sd_b, length.out = 200),
  density    = dnorm(x, mean = mean_b, sd = sd_b),
  conference = "B",
  attendance = conf_b_attendance
)

standardize_curves <- bind_rows(conf_a_curve, conf_b_curve)
standardize_curves

```

> OUTPUT

```

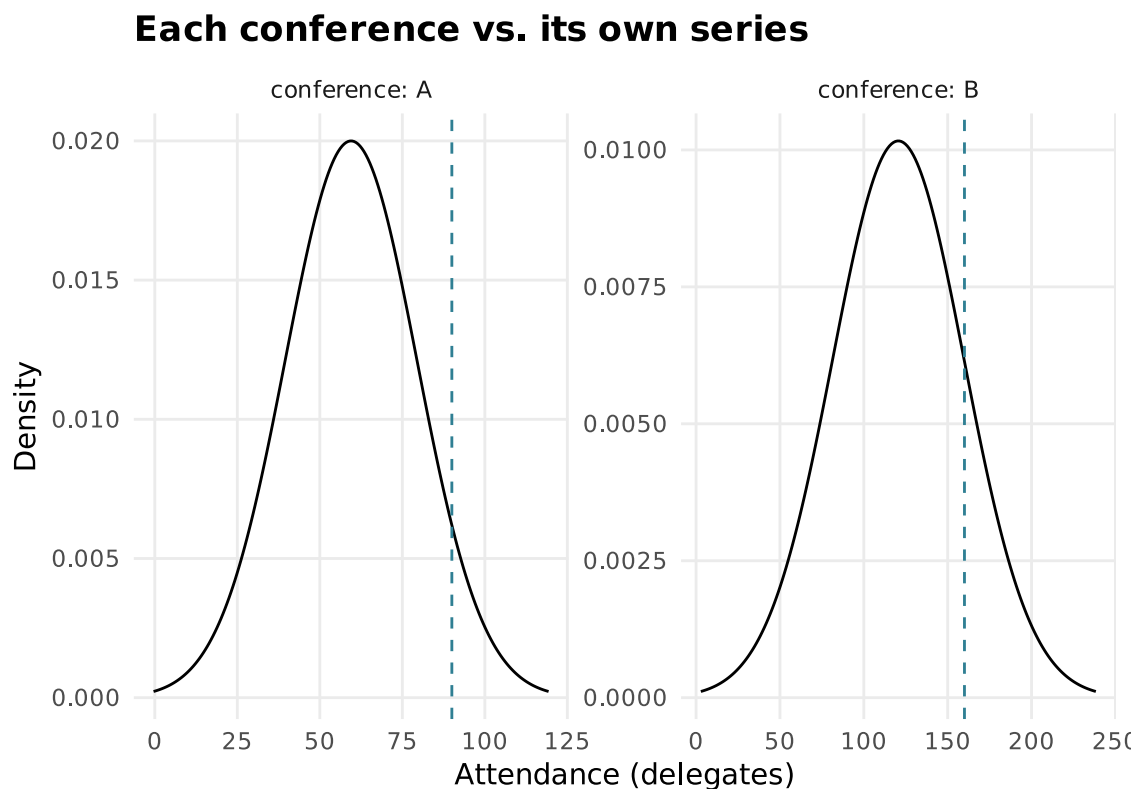
# A tibble: 400 x 4
      x density conference attendance
  <dbl>   <dbl> <chr>         <dbl>
1 -0.372 0.000222 A             90
2  0.229 0.000243 A             90
3  0.831 0.000266 A             90
4  1.43  0.000290 A             90
5  2.03  0.000317 A             90
6  2.63  0.000345 A             90
7  3.24  0.000376 A             90
8  3.84  0.000409 A             90
9  4.44  0.000445 A             90
10 5.04  0.000483 A             90
# i 390 more rows

```

```

standardize_curves |>
  ggplot(aes(x = x, y = density)) +
  geom_line() +
  geom_vline(aes(xintercept = attendance), color = course_primary, linetype = "dashed") +
  facet_wrap(~conference, scales = "free", labeller = label_both) +
  labs(
    title = "Each conference vs. its own series",
    x      = "Attendance (delegates)",
    y      = "Density"
  ) +
  theme_intror()

```



Try it: a tighter invitation cap

Your task: what's the probability of overflowing a 100-seat venue if the summit chair caps invitations at 110 instead of 120 (same 85% attendance rate)?

```
1 - pbinom(q = 100, size = 110, prob = 0.85)
```

> OUTPUT

```
[1] 0.02442528
```

Try it: standardizing a small conferences tibble

Your task: two more conferences, two more series — using a tibble and `mutate()`, compute z-scores to find which one had the stronger year relative to its own history: Conference C (45 delegates, series average 30, sd 10) or Conference D (210 delegates, series average 150, sd 50)?

```
tibble(
  conference = c("C", "D"),
  attendance = c(45, 210),
  series_mean = c(30, 150),
  series_sd = c(10, 50)
) |>
mutate(z = (attendance - series_mean) / series_sd)
```

>_ OUTPUT

```
# A tibble: 2 x 5
  conference attendance series_mean series_sd     z
  <chr>          <dbl>        <dbl>    <dbl> <dbl>
1 C              45          30        10  1.5
2 D             210         150        50  1.2
```

Try it: CLT at a larger sample size

Your task: repeat trial 1.b (the Normal delegate-age population, $n = 400$ per survey) at $n = 900$ per survey. Does the sampling distribution get tighter still, and does `var(trial)` land close to the theoretical `var(pop)/900`?

```
sample_size <- 900
trial_1c <- expand_grid(
  batch      = 1:n_batches,
  participant = 1:sample_size
) |>
group_by(batch) |>
mutate(age = sample(delegate_ages, size = sample_size, replace = TRUE)) |>
summarize(sample_mean = mean(age), .groups = "drop")

trial_1c |> summarize(var_sm = var(sample_mean))
```

>_ OUTPUT

```
# A tibble: 1 x 1
  var_sm
  <dbl>
1 0.0265
```



```
tibble(population_variance = var(delegate_ages)) |>  
  mutate(theoretical_var = population_variance / 900)
```

> OUTPUT

```
# A tibble: 1 x 2  
  population_variance theoretical_var  
      <dbl>         <dbl>  
1          23.0         0.0255
```